most recommended programming languages
- Even today C helps beginners learn and understand computer logic and programming fundamentals.

Features of C.

① Simple and portable language
'C' was develop so that it incorperated functions to simplify mathematical and logical implementation and set of rules or procedures to use them, thy approach made it simple and easy to understand.

② Middle level language
It combines features of both high level & low level language, resulting in faster execution and smoother application development hence C became first middle level language

③ Modularity
The whole 'C' programm can be divided into ~~Indivision~~ visual & functional units, it means every indivisual function in the programm acts as independent unit to perform specific operation.

④ Rich Library
C language comes with built in libraries and functions, this prewritten header file help users to borrow and run mathematical and logical operations directly from library which helps them to write simple & complex programm easily

⑤ Procedural language: C follows top down approach for software developments. C program is divided into smaller unit called functions.

⑥ Easy to extend: The extandable nature of C allows developers to add additional functions to the 'C' library and also it quickly adopt new features.

⑦ Built in operators:-
C language contain a wild range of built in operators that are used for different scenarios such as arithmatic, relational, bitwise and logical operators.

IDM Imp for

Structure of C.

① Documentation Section (comment)
② Link section. (header files)
③ Definition Section (symbolic constant)
④ Global declaration section. (global variable)
⑤ main() function section (int/void main)
{
Declaration part (local declaration)
Executable part (body of the program)
}

⑥ Subprogram Section
Function 1
Function 2
Function 3
-
-
-
Function n

(1) Documentation section / comment section

1. Comment is a text that compiler ignores but that is useful for programmers
2. This section consist of comment lines.
3. It is used to give the name / description of a particular line on the program
4. Documentation section helps anyone to get an overview of the program.
5. Two types of comments are
   (i) Single line comment
   (ii) Multi line comments

→ (i) Single line comment

- comments can be given in single line by using "//".
  The general syntax is
- // Single Text line.

For example,

1. // Add two numbers (Headings of the programm)
2. c = a+b; // store the value of a+b in c.
   (comments to explain the given statements)

→ (ii) Multi line comment

comments can be given in multiple lines starting by using "/*" and end with "*/"

general syntax

```
/*Text line 1
Text line 2
Text line 3*/
```

For ex:-

```
/* Write a C programm to find the sum and average
of five numbers. */
```

(2) Linker Section

1. The link section consists of the header files of the functions that are used in the program
2. It provides instructions to the compiler to link functions from the system library

ex:

```
#include <stdio.h>
#include <conio.h>
#include <math.h>   [used in sqrt]
```

(3) Definition (Definition) section

1. All the symbolic constants are written in definition section.
   Macros are known as Symbolic constants.
2. By using definition section we can define a variable with its value. For this purpose define statement is used

General syntax.

- #define variable name value.

ex:

- #define PI 3.142
- #define A 100
- #define NAME "Dinesh"

(4) Global Declaration section

1. Global variables that can be used anywhere in the program are declared in this section
2. User defined function also declared in this section.
3. We can declare some variables before starting of main function or outside the main function these variables are named Global variables.

general syntax.

- data-type v1, v2, v3 ... vn;

Example:

- int a,b,c;

(5) Main() function Section

1. It is necessary to have one main() function section in every C program.
2. This section contains two parts
   (1) Declaration
   (2) Executable part
3. The declaration part declares all the variables that are used in executable part.
4. Each statement in the declaration & executable part must end with a semicolon (;)
5. Main function is where a program starts its execution.

→ (i) Local declaration section (declaration)
The variables and its data type declare inside the function block and such variables remain till the end of the block.

Ex:

- Int a;
- Float b;

→ (ii) Executable statements section
Executable statements are written inside the main function or any other function. These statements can be expressions, input, output, & functional statements and so on

(6) Subprogram / User Defined section

1. This section contains all the user defined functions that are used to perform a specific task.
2. These user defined functions are called in the main() function.

Example 1:-

(1) write a C program to display your name

```
// C program to display name. (// comment section)
or /* C program to display name */
#include<stdio.h>                    // link section
int main()                           // main function
{
printf("My name is Manasa");
return 0;
}
```

(2) write a C program to find (To add) two numbers.

```
// C program to add two numbers.
#include<stdio.h>                    // link section
int main()                           // main function
{
int a,b,c;                           // local declaration
printf("enter value for a\n");
scanf("%d",&a);
printf("enter the value for b\n");
scanf("%d",&b);
c=a+b;                               // executable part
printf("addition of two number is %d",c);
return 0;
}
```

o/p
enter value for a 10
enter value for b 30
addition of 2 number is 40

③ Write a c programm to find area of ckt

```
// C programm to find area of ckt (comment section)
#include<stdio.h>; //(linker section)
# define pi 3.142 //(declaration section)
float a; //(global declaration)
int main( ) // (main function section)
{
int r; // (local declaration section)
printf ("enter radius of the cricle \n");
scanf ["%d", &r] ;
a = pi *r *r ;
printf ("\n area of the circle %f", a) ;
return 0;
}
```

(Declaration part)

```
#
// C programme
```

O/P

enter the radius of circle 3
area of the circle 28.278000

④ write a c programm to find sum and average of 5 subjects.

```
// C program to find sum and average of 5 subject
# include<stdio.h>
int main( )
{
int a, b, c, d, e, sum, average;
printf (" enter the sum of 5 subject \n");
scanf (" %d; %d, %d, %d, %d, &a, &b, &c, &d, &e)
   sum = a+b+c+d+e ;
printf (" enter the average of 5 subject \n");
scanf (" %d, %d, %d, %d, %d , &a, &b, &c, &d &e);
   average = a+b+c+d+e
                5
printf ("\n sum = %d", sum);
printf ("\n average = %d, average);
return 0;
}
#include<stdio.h>. // link section
int main() // main function.
{
int s1, s2, s3, s4, s5, sum, average; //local declaration
printf ("enter marks of subject 1 \n");
scanf ("%d", &s1);
printf ("enter the marks of subject 2 \n");
scanf ("%d", &s2);
printf ("enter marks of subject 3 \n");
scanf ("%d", &s3);
printf (" enter marks of subject 4 \n")
scanf ("%d", & s4);
printf ("enter marks of subject 5 \n")
scanf ("%d", &s5);
```

```
sum = s1+s2+s3+s4+s5;          // executable part
average = sum/5;
printf("\n sum = %d", sum);
printf("\n average = %d", average);
return(0);
}
```

o/p
enter the marks of subject 1
20
enter the
30
40
50
60
sum = 200
average = 40

Creating And Executing C Program

- writing source code.
- saving the code.
- compiling the source code.
- executing the c program.

① Writing source code – source code is actually a set of instruction which a programmer write in an editor.

- these instructions are written according to the syntactical rules of C language.

② Saving the code :- The source code file must be saved with the file extension .C
for example - sample.c

③ Compiling the source code :- A source code is written in alphanumeric form which computer cannot understand.
It must be converted into binary form.
The process of translating high level language program into machine level language program is called compilation
when compilation is completed a new file is created in 'C' called object file. It's file

- extension .obj and it is created by the compiler.

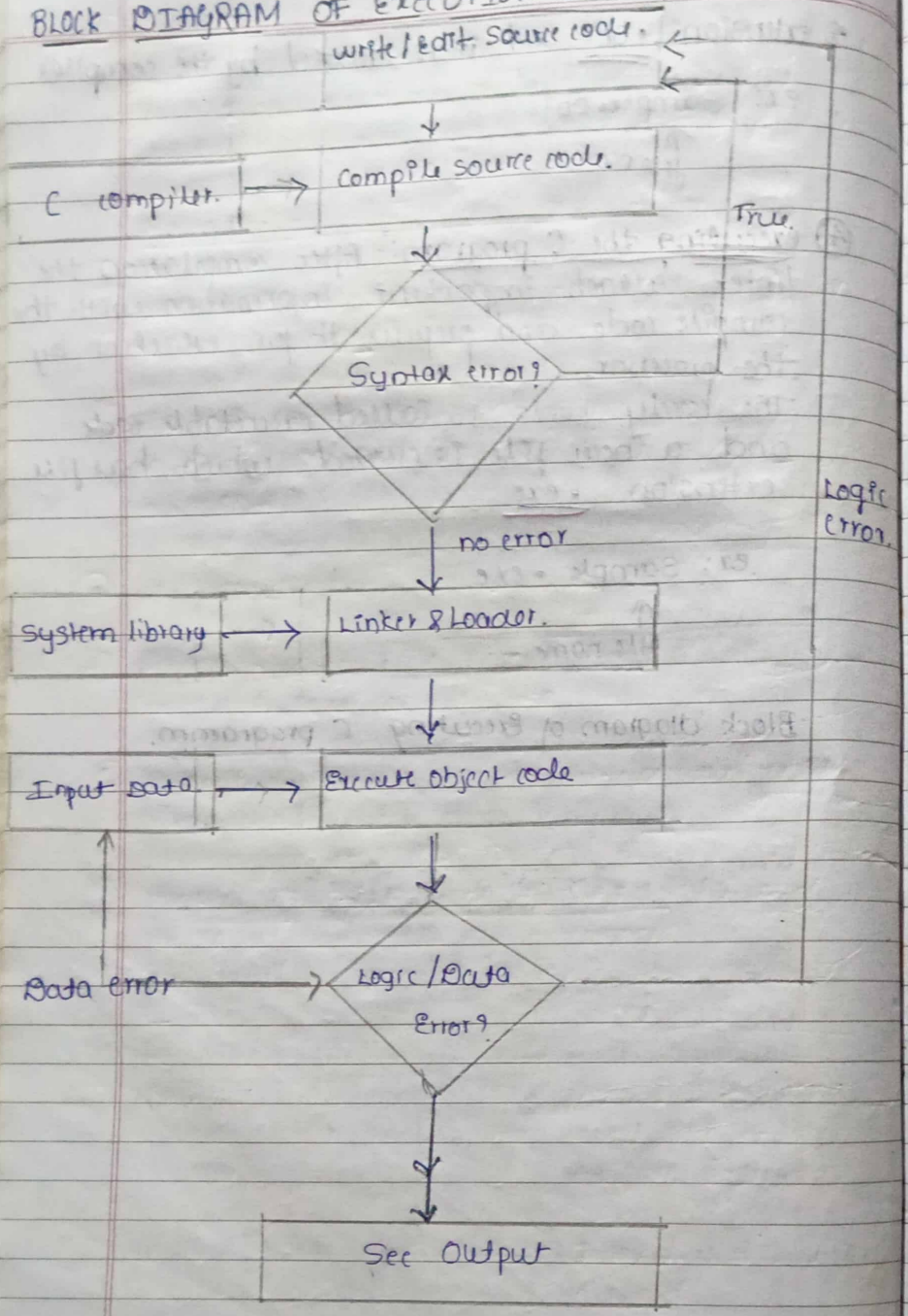
ex: sample.obj
↑ file name.

④ executing the C program :- After compilation the linker attach important information with the compile code and prepare it for execution by the processor.
The ready code is called executable code and a new file is created which has file extension .exe

ex: sample.exe
↑ file name

Block diagram of Executing C programm.

BLOCK DIAGRAM OF EXECUTION OF C



Chapter - 2

② BASIC CONCEPTS.

C character set

- character set means a valid set of characters that language recognise,
- Character sets are classified into three groups

→ 1. Alphabets. [lower case or upper case [A-Z, a-z]
→ 2. Digits [0 to 9]
→ 3. Special symbols.

V.V. Imp

C Tokens

A token is a smallest individual unit in a programm

C has following tokens

① Keywords ⇒ char
② Indentifiers → int score = 90 [score is an identifier]
③ Constants a = 10 / literal
④ Variable a, b
⑤ Operators + - * /

① Keywords :-

Keywords are Predefined reserved identifiers that have special meanings to the compilers. The programmer is not allowed to change its Meaning.

List of C keywords (ex):

auto	goto	struct	sizeof
break	if	switch	
char	int	enum	
case	static	extern	
const	do	float	
continue	double	for	
default	else	static	
void	short	signed	

② Identifiers

Identifiers are used as the general terminology for the names of variables, functions and arrays. [Programming element - arrays, variable, functions]

Rules for naming Identifiers.

① The first character of the identifier must be letter of the alphabets (upper or lowercase) or an underscore ('_')

② The rest of the identifier name can consist of letters (upper or lower case), underscores ('_') or digits (0-9)

③ The name cannot starts with digits

④ Identifier names are case sensitive
For ex: A & a are not same

⑤ A declared keyword can not be used as identifiers/variable name

Some examples of C Identifiers

	Name	Remark
①	_A9	valid
②	Temp.var	Invalid as it contains special character other than the underscore.
③	Void	Invalid as it is a keyword

③ Constants

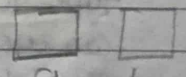
Eg:
- Constants refer to fixed values that the program may not alter.
- Any value declared as const can not modified by the program i.e. It wont change during the time of execution.
- An identifier that represents a constant value throughout ~~throughout~~ the life of the program is known as symbolic constants
- Ex: const int a=10; → variable / constant

keyword — Symbolic constant

There are Four types of constants
① Integer constants
② Floating constants
③ character constents
④ String constents

④ Variables

Variables are containers for storing data values
variable represent the name of the memory location



Syntax

datatype variable_name_list ;

For ex:
- int a;
- float b;

Initializing A variable.

Syntax:- datatype variable_name = value;

For ex: int a = 100;

(5) Operator

An operator is a symbol that operates on a value or a variable.

ex; + is an operator to perform addition
- " " " subtraction
* " " " multiplication
/ " " " division.

DATA TYPES

- Data types specify how we enter data into our programs and what type of data we enter
- C language has some predefined set of data type to handle various kinds of data that can use in our program.
- These data types have different storage capacities.
- C data types are broadly classified into 3 type

① Fundamental or built in datatype
(i) int.
(ii) float < Float, double.
(iii) Char.
(iv) void.

② Derived data type.
(i) Array → any no of variable in one sentence.
(ii) Function → perform specific task
(iii) Pointer. * → store addrss of another variable.
(iv) Reference. → same operation diff name / another name

③ User defined data types.
(i) structure
(ii) union
(iii) enumeration

① Fundamental or built in data type

Fundamental data types are basic built in datatypes of c programming language

(i)(int) Integer data type.

- Integer data type is used to store a numeric value.
- Integer data type is used to declare whole number (no decimal value) that can hav 0 or -ve or +ve value.
- Integer variable is denoted by int keywor
- The storage size of integer is 2 or 4 or 8 bytes it depends upon the platform of the system
- The range of number we can store from -32768 to 32767
- ex:- 0, 8, -5

Size range of integer Type on 16 Bit Machi

Type	Size (bytes)	Rang
int or signed int	2	-32768 to 3276
unsigned int	2	0 to 65535

(ii) Floating point type.

- Floating data type is used to declare decimal number (no whole value). that can have positive and negative value.
- Floating point is denoted by float key word int
- The storage size of the following points is 4 bytes. but it depend upon the platform of the system
- Ex: 24, 35
- There are two types of floating

→ (1) Float

This keywords is allows to store single precision floating number

They occupy 4 bytes of memory space

The range is 3.4×10^{-38} to $3.4 \times 10^{+38}$

→ (2) Double

This keyword allows to store double precision floating number. They occupy 8 bytes of memory space. The range is

1.7×10^{-308} to 1.7×10^{308}

Size and Range of floating type on 16 bit machine

Type	size (bytes)	Range.
Float	4	3.4E-38 to 3.4E+38
Double	8	1.7E-308 to 1.7E+308.

(iii) Character Type only one character

- character types are use to store character value
- The character is denoted by **char** keyword.
- It is use to store only one character using the char data type.
- They occupy 1 byte of memory space.
- Image is -128 to 127.

Example : 'A', 'C'

Size and Range of Char type on 16 bit

Type.	Size (byte)	Range
char or signed char	1	-128 to 127.
unsigned char	1	0 to 255

(iv) Void Type.

- Void is a data type (It is also key word) means No value or nothing
- This is usually use to specify the type of functions which returns nothing.
- It is mostly used in function identification for not return anything
- You cannot creat a variable as void type

ALM

	Integer	Float	Double	Character
size	2	4	8	1
Range 16 bit prolly	-32768 to +32767	3.4×10^{-38} to 3.4×10^{38}	1.4×10^{-308} to 1.4×10^{308}	-128 to 127.

Class work

Date 16/10/24

C.W

Derived Data type

derived from fundamental data type

array - collection of homogeneous elements which have same name & data type.

any no of variable in array → int a [100]

array

int a [4] → array size

index value → 0 to n.

a[4] 2030	2
a[3] 2028	2
a[2] 2026	2
a[1] 2024	2

separate index value

continuous memory location

discontinuous memory location

static memory → not change 20 bytes → word is byte

pointer

dynamic " → heap of pointer (change)

a 2030 [ ] 2 int *ptr

int * ptr → 2035 (address of another

char name[7]

Manasa

static

---

② Derived data type.

Defn • Data types that are divided from the built-in data types are known as derived data types

The various data types provided by C are

1. arrays
2. functions
3. Pointers
4. reference

ARRAY

Array defn: An array is a set of elements of the same data type that are referred to by the same name,

- All the elements in an array are stored contiguous (one after another) memory locations and each element is accessed by unique index or subscript value or index value.
- The subscript value indicates the position of an element in an array
- Syntax: dataType array Name [array size];

Ex: - int mark[5];

mark0 mark[1] mark[2] mark[3] mark[4]

char name[50]; → character array

FUNCTION

Function defn:- Function is a self contained program segment that carries out a specific well defined task

- In C every program contains one or more functions which can be invoked from other parts of a program if required

POINTER

Pointer defn : A pointer is a variable that can store the memory addrss of another variable.

- Pointers allow using the ~~dem~~ memory dynamically
- That is with the help of pointers memory can be allocated or de-allocated to the variables at run-time, thus making a program more efficient

Syntax - datatype *var_name; (no space)

Ex:- int *ptr;

REFERENCE

defn :- A reference is an alternative name for a variable
- that is a reference is an alias for a variable in a program
- A variable and its reference can be used interchangebly in a program as both refer to the same memory location.
- Hence changes made to any of them (say a variable) are reflected in the other (or a reference)

dont have string data type in C → instead char

⑨ User Defined Data type. → defined by user

group of variable diff data type & name

size of structure considered by adding size.

name - 50 byte.	name - 50
cit no - 2	cit no - 2
float - 4	float - 4
56 byte	50 byte
↑ structure	↑ union (largest size)

User defined data types

User defined data types in C is a type by which the data can be represented..... Hence the data types that are defined by the users are known as user defined data types

STRUCTURE

A structure creates a datatype that can be used to group items/elements possibly different types into a single type.

'Struct' keyword is used to create a structure

Example,

Syntax →. struct structure_Name struct Person

{ {

datatype member 1 : (50) char name[50];

data type 2 : (2) int cit NO;

data type 3 : (4) float salary

data type N.

}; }

The size of the structure person is 50+2+4=56 bytes

UNION

- Union is user defined data types all the members of union share same memory location
- The size of the union is decided by size of largest member of union.
- If you want to use same memory location for two or more members, Union is best for that.

Syntax.

```
Union union Name
{
data type number1;
data type number2;
 ''      ''  3;
 ''      ''...N;
};
```

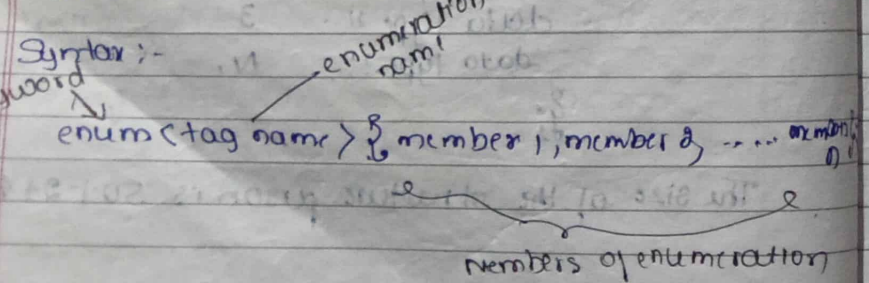
Example.

```
union result
{
int marks;
char grade;
float percent;
. . .
};
```

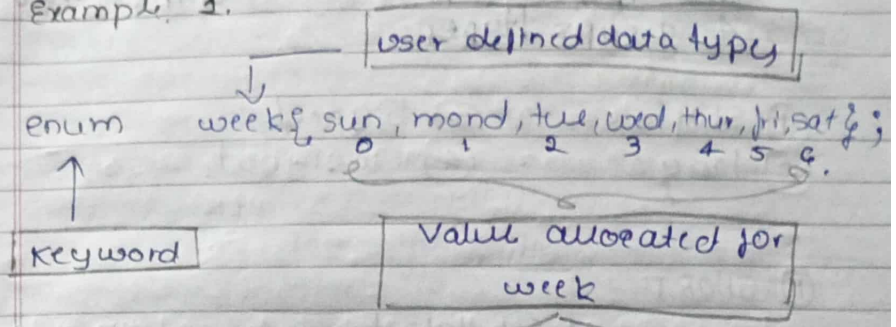
Size of union result is 4 bytes since percentage member is having largest size among all

ENUMERATION

- An enumeration is user defined data types that consist of integer constants
- To define an enumeration keyword enum is used.
- Syntax:-



Example 1.



Example:- enum country {US, UN = 3, India, china};
0 3 4 5

Data type Modifiers. - change the property of data type

short :- To make constant bytes for all bit processor. 2/4/8 bytes in all type of processor

long :- increase size of integer Integer size 2+2 adding 2 more bytes. 2+2=4.

long int - 4
long double - 10.

signed - signed int a=10
a = ±10 (both value)

unsigned - only consider +ve.

Data type Modifiers

In C language Data type Modifiers are keywords used to change the properties of current properties of data type.

Modifiers are prefixed with basic data types to modify (either increase or decrease) the amount of storage space allocated to a variable

Data types modifiers are classified into 4 groups / types

1. Short
2. long
3. Signed
4. Unsigned.

① SHORT

In general int data type occupies different memory space for different operating system. to allocate fixed memory space, short keyword can be used.

Syntax:- short datatype variablename;

Example:- short int a; ... > occupies 2 bytes of memory space in every operating system.

② LONG.

This can be used to increase size of the current dat type to 2 more bytes, which can be applied on int or double data type

For ex: int occupy 2 byte of memory. If we use **long** with integer variable then it occupy 4 byte of memory

Syntax: Long data type variablename;

Ex : long int a; ...... < by default it represent long it
long a;
long double a.

next thursday submission
November - 7
exam syllabus

SIGNED.

This keyword accepts both **negative & +ve** value and this is default properties of data type modifiers for every data type.

Examples: int a=10;
int a = -10;
signed int a = 10;
signed int a = -10;

UNSIGNED

This keyword can be used to make the accepting values of data type is +ve data type

Ex:

- unsigned int a = 100; // right statement
- unsigned int a = -100; // wrong

Program

```
// Area of circle
#include<stdio.h>
define pi 3.142
float areac;
int main( )
{.
int r;
printf("enter the radius of circle \n");
scanf("%d", &r);
area c = pi*r*r;
printf("area of circle =%f", area c);
return 0;
}.
```

```
// Area of Δle of given 3 sides
#include<stdio.h>
#include<math.h>
float areat, s;
int main( )
{
int a,b,c;
printf("enter the value of a\n");
scanf("%d ", &a);
printf("enter the value of b\n");
scanf ("%d", &b);
printf("enter the value of c\n");
scanf ("%d", &c);
s = (a+b+c)/2
areat = sqrt s*(s-a)*(s-b)*(s-c)
printf ("area of triangle = %f", areat);
return 0;
}.
```

Lab Program.-1

Q. Write a C program to find area of o^le and area of Δ^le given three side.

Area of o^le $= \pi r^2$

Area of Δ^le when 3 sides given $= \sqrt{s(s-a)(s-b)(s-c)}$

semi perimeter ← $s = \frac{(a+b+c)}{2}$

```
// C program to find area of circle and area of triangle
   given three side //
#include<stdio.h>          ] linker sectio
#include<math.h>           ]
#define pi 3.142           → definition section
int main( )                → main function
{
int r, a, b, c;            → local declaration
float areac, areat, s;
printf("enter radius \t");
scanf("%d", &r);
area c = pi*r*r;           → executable part
printf("Area of circle=%f", areac);
printf("\n enter 3 sides \n");
scanf("%d %d %d", &a, &b, &c);
 s = (a+b+c)/2;
areat = sqrt(s*(s-a)*(s-b)*(s-c));
printf("Area of triangle = %f", areat);
return 0;
}
```

enter radius -1 Area of circle = 3.142000 1 1.6 85.976738	o/p. Enter radius 12 Area of circle = 452.447998 enter 3 sides 14 15 12 Area of triangle = 69.282036.

UNIT-2

Chapter -3.

③ INPUT / OUTPUT FUNCTION (STATEMEN)

I/O FUNCTION.

I/O function links b/w user & function. gets( );

I/P	O/P.
→ scanf( )	→ printf( )
→ getchar( )	→ putchar( )
→ getch( )	→ putch( )
→ getche( )	→ puts( )
→ gets( )	→

I/P O/P classified
① formatted I/O function → scanf → use format specifier
② unformatted O/P function → printf

control string or means format specifier

variable – name of memory location

scanf – taking data from user with the help of input

%s – specification

%d, %f, %c } & for I/P's — & not required.

Introduction.

- In C, the standard library stdio.h provides functions for input and output
- The instructions #include<stdio.h> tells the compiler to search for a file named stdio.h and places its contents at this point in the program.
- The content of the header file become part of the source code when it is compiled
- Input/output functions are links between the user and the terminal.
- Input functions are used to read data from keyboard are called standard input functions.

ex. – Scanf( ), getchar( ), getche( ), getch etc.

- output functions are used to display the result on the screen are called standard output functions

ex. – printf( ), putchar( ), putch( ), puts( ) etc.

The input/output functions are classified into two types

→ (1) Formatted functions
(2) Unformatted functions

(1) FORMATTED I/O FUNCTIONS

- Formatted I/O functions are used to take various input from the user and display multiple outputs to the user
- These types of I/O functions can help to display the output to the user in different formats using the format specifiers
- These I/O support all data types like int, float, char and many more.

Why they are called formatted I/O?

- These functions are called formatted I/O functions because we can use format specifiers in these functions and hence we can format these functions according to our needs

Types of formatted I/O function

→ (1) Input function : scanf( )
(2) Output function : printf( )

(1) Input function - scanf( )

scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any datatype like integer, character, string, float etc.

- scanf() stands for scan formatted
- This function is declared in stdio.h (header file) thats why it is also predefined function
- In scanf function scanf() function we use &(address-of operator) which is used to store variable value of memory location.

The general form/Syntax of scanf() is

scanf("control string", arg1, arg2, ... argn);

- control string → format in which data is to be entered
- arg1, arg2 → location where the data is stored, → preceded by ampersand(&)

List of some format specifiers (control strings)

Format specifier	Type	Description
%d	int/signed int	Used for I/O signed integer value.
%c	char	Used for I/O character value.
%f	float	Used for I/O decimal floating-point value.
%s	string	Used for I/O string/group of characters.

Example 1

```
#include<stdio.h>
void main()
{
int n;
char ch;
printf("Enter a number:");
scanf("%d",&n);
printf("enter a character:");
scanf("%c",&ch);
printf("\n Number:%d\t character:%c",n,ch);
getch();
}
```

Example 2:

```
#include<stdio.h>
#include<conio.h>
void main(){
char string[10];
printf("Enter Your Name:");
scanf("%s",string);
printf("My Name is %s",string);
getch();
}
```

Problem

① Write a program to read student information like name, roll number, semister, course and percentage.

```
#include<stdio.h>
#include<math.h>
main();
int rollN;
string name[50];course;
float percentage;
printf("Enter the name of student");
scanf("%c",&name)
printf("enter the roll number");
scanf("%d",&rollnumber);
printf("enter the semister\n");
scanf("%s",&semister);
printf("enter the course\n");
scanf("%s",&course);
printf("enter the percentage\n");
scanf("%f",&percentage);
printf("name=%s roll number=%d
semister=%s course=%s percentage=%
,name,rollnumber,semister,course,percentage
return 0;
}
```

Formatted Output – printf()

- Formatted output refers to the data that has been arranged in particular format
- printf() function is used in C program to display any value like float, integer, character string, etc on the console screen
- It is pre-defined function that is already declared in the stdio.h (header file)

Syntax1: To display any variable value.

printf("Format Specifier", var1, var2, ..., varn);

syntax 2: To display any string or a message

- printf("Enter the text which you want to display");

12/10/2024

Unformatted Input/Output functions

- Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype.
- Doesn't allow user to read or display data in desired format
- These library functions basically deals with a single character or a string of characters
- The functions getchar(), putchar(), gets(), puts(), getch(), putch() are considered as Unformatted functions.

Why they are called Unformatted I/O?

- These functions are called unformatted I/O functions because we cannot use format specifiers in these functions and hence, cannot format these functions according to our needs

① getch() → declared in conio.h.

- getch() function reads a single character from the keyboard by the user but does not display that character on the console screen and immediately returned without pressing enter key
- This function is declared in conio.h (header file) getch() is also used for hold the screen

Syntax

- getch();
  or
- variable-Name = getch();

② getchar() → stdio.h

- Reads a character from a standard input device.
- It takes the form
  character_variable = getchar();
- character_variable is a valid c char type variable.
- When this statement is encountered the computer waits until enter key is pressed and then assigns this character to character_variable.

③ getche()

- Reads single character the instant it is typed without waiting for the enter key to be hit
- getche() displays the character when entered
- General form
  - character_variable = getche();

unformatted → I/P & O/P

I/P.
- getch()
- getche() — for char
- getchar()
- gets() — for string

O/P.
- putch()
- putchar() — for char
- puts() — for string

getch() → only takes input, donot display

getche() → takes input and display on screen

getchar() → takes multiple input but display first input

gets →

④ gets()

→ Used to read string of text, containing whitespaces until a new line character is encountered

→ General form - gets(string_variable);

O/P.

① putchar() (stdio.h) → prints single characters

→ Displays a character to the standard output device

→ Its form - putchar(character_variable);

→ where character_variable is a char type variable containing character

Ex:

```
#include<stdio.h>
#include<conio.h>
void main()
{
char gender;
printf("Enter gender M or F");
gender=getchar();
printf("Your gender is:");
putchar(gender);
getche();
}
```

② Puts() <stdio.h> → prints a string with new line

→ Used to display the string onto the terminal

→ General form puts(string_variable);

Ex:

```
#include<stdio.h>
#include<conio.h>
void main()
{
char name[20];
printf("Enter your name:");
gets(name);
printf("Your Name is:");
puts(name);
getch();
}
```

③ putch() conio.h

→ The function putch() prints a character onto the screen

→ General form - putch(character_variable);

→ character variable is a char type.

Note: These 3 functions are defined under the standard library function conio.h

Ex:

```
#include<stdio.h>
#include<conio.h>
void main()
char ch1, ch2;
printf("enter 1st character: ");
ch1 =getch();
ch2=
printf("\n enter 2nd character;");
ch2 =getche();
printf("\n first character;");
putch(ch1);
printf("\n second character:");
putch(ch2);
getch();
}
```

Write a C program to read and print information like name, gender, birth place or whether student or not. using unformatted function.

```
#include<stdio.h>
#include<conio.h>
void main()
char gender, student or not;
char name[10], birth_place[10].
printf("Enter the gender");
gender = getchar();
printf("your gender is:");
printf("enter whether they are s");
s = getchar();
printf("they are s");
```

```
putchar(s)
printf("enter your name");
gets(name);
printf("your name is");
puts(name);
printf("enter the birthplace");
birthplace = getchar();
printf("your birth_place is");
putchar(birthplace);
getch();
}
```

print("your name")
puts(name)

```
#include<stdio.h>
#include<conio.h>
void main()
{
char name[22], place[24], gender, student;
printf("enter name\n");
gets(name);
printf("enter gender M or F\n");
gender = getche();
printf("enter place\n");
gets(place);
printf("enter student Y or N\n");
student = getche();
printf("\ngender is");
putch(gender);
printf("\nplace is");
puts(place);
printf("\nstudent");
putch(student);
return 0;
}
```

name
gender
student

Chapter - 4

OPERATORS & EXPRESSION

a+b → operator; operand

+ is operator

a & b are operand

Types

1. Unary → one operand & operator (unary minus, increament, decrement operator)
2. binary → two
3. Ternary → three. (conditional operator)

① Unary — one operator on one operand

(i) Unary Minus — changes +ve operand to -ve, -ve to +ve.

(ii) Increament — increase value by 1

numbers - 150 250

Preincreament (Prefix): ++a ↓ increase instant

Postfix (post increament): a++ ↓ increase after use.

++a, a=100, print 101, 102 ....

a=100, a++, print 100, 101, ...

(iii) Decreament operator

Predecreament: a=100, --a, 99

Postdecreament: a=100, a--, 100

② Binary — 2 operator on 2 operand

% → Reminder gives.

a%b → a modulo b remainder

(1) Arithmatic operator → addition, multi, sub, division (returns quotient), modular division (modulo) (returns reminder)

② Relational operator — compare expression: greater than equal to, smaller than equal to, is equal to, not equal

a=10, a=b

a=10, b=20

a == b

10 == 0 (10 = 0)

Bitwise — & — a&b — convert to binary

5<<1 = 10

1010 → 8 4 2 1 → 10

~5 → 101 → 010

5771

③ logical operator — && logical AND, || logical OR, ! logical NOT

④ Assignment operator — simple =, compound (arithmatic +=)

compound: a+=10, a=a+10

Introduction

- In C language operators are symbols that represent operations to be performed on one or more operands.
- They are the basic components of the C programming
- An operator in C can be defined as the symbol that helps us to perform some specific mathematical, relational, bitwise, conditional, or logical computations on values & variables.
- The values and variables used with operators are called operands, so we can say that the operators are the symbols that perform operations on operands

Types: operators are classified into 3 category

→ ① Unary operator

→ ② Binary operator

→ ③ Ternary operator

→ ① Unary operator.

Unary operators in C are single operators that work on a single operand.

Operator	Symbol	Description
Unary minus	-	Returns opposite sign operand
Increament operator	++	Increaments the value of an operand by 1
Decreament operator	--	Decreaments the value of operand by 1

(1) UNARY MINUS (-) OPERATOR

- Unary Minus(-) operator is used to change the sign of an operand.
- It changes a positive operand to negative and a negative operand to positive.

Ex:
```
#include<stdio.h>
int main()
{
int num, inverseNum;
printf("Enter a number");
scanf("%d", &num);
inverseNum = -num
printf("original number %d \n", num);
printf("opposite of given number %d \n", inverseNum);
return 0;
}
```

o/p
2
opposite of given number -2

Increament (++) operator

- Increament(++) operator used to increase the value of an operand by one.
- There are two types of increament(++) operator
  - → (1) Prefix increament operator
  - → (2) Postfix increament operator.
- The difference blw prefix & postfix increament operator is that prefix increament increase the value of operand instantly, however postfix increament operator increase the value after it is used

Ex:
```
#include<stdio.h>
int main()
{
int num, num2;
printf("Enter num and num2 values");
scanf("%d%d", &num, &num2);
printf("Pre-increament variable num %d \n", ++num);
printf("Post-increament variable num2 %d \n", num2++);
return 0;
}
```

o/p
enter num and num2
100 , 101
Preincreament of num 101
Post increament 101, 102

100 100
Preincreament num1 101
Post inc num2 100

Decreament (--) operator

- decreases the value of an operand by one.
- decreament operator also has 2 types
  - → (1) Prefix decreament operator : decreases the value of an operand by one instantly
  - (2) Postfix decreament operator : decreases the value of an operand by one after it is used

Ex:
```
#include<stdio.h>
int main()
{
int num, num2
printf("enter num and num2 values \n");
scanf("%d%d", &num, &num2);
printf("Pre decreament variable num%d \n", --num);
printf("Post decreament variable num2 %d \n", num2--);
return 0;
}
```

o/p
pre decreament num1 119
post decreament num2 120

→ ② BINARY OPERATOR

Binary operators in C are operators that work on two operands to produce a new value.

5 types of Binary operator

① Arithmetic Operators
② Relational operators
③ Logical operators
④ Assignment operator
⑤ Bitwise operators

~~ARI~~

① Arithmetic operators

Arithmetic operators are used for numerical calculations (or) to perform arithmetic operations like addition, subtraction, etc.

Operator	Description	Example
+	Addition	a+b
-	Subtraction	a-b
*	multiplication	a*b
/	Division	a/b
%	modulus (modular division)	a%b

Ex:

```
#include<stdio.h>
void main(){
int a,b;
printf("enter a and b value");
scanf("%d%d",&a,&b);
printf("a+b=%d\n",a+b);
printf("a-b=%d\n",a-b);
printf("a*b=%d\n",a*b);
printf("a/b=%d\n",a/b);
printf("a%b=%d\n",a%b);
getch();
}
```

o/p

```
enter value of a and b
10
20
a+b=30
a-b=-10
a/b=0
a*b=200
a%b=10
```

② Relational

Relational operators are used for comparing two expression such as greater, lesser, equal and not equal.

Operator	Description	Ex.	a=10, b=20	o
<	less than	a<b	10<20	1
<=	less than (or) equal to	a<=b	10<=20	1
>	greater than	a>b	10>20	0
>=	greater than (or) equal to	a>=b		
==	is equal to	a==b		
!=	not equal to	a!=b		

Ex:

```
#include<stdio.h>
void main(){
int a,b;
printf("enter a and b value");
scanf("%d%d",&a,&b);
printf("%d\n",a<b);
printf("%d\n",a<=b);
printf("%d\n",a>b);
printf("%d\n",a>=b);
printf("%d\n",a==b);
printf("%d\n",a!=b);
getch();
}
```

o/p

a=20, b=10

a<b : =0
a<=b =0
a>b =1
a>=b =1
a==b =0
a!=b =1

③ Logical operators

Logical operators are used to combine 2 (or) more expressions logically

operator	Description	Ex
&&	logical AND	(a>b)&&(a<c)
||	logical OR	(a>b)||(a<=c)
!	logical NOT	!(a>b)

Ex:

```
#include<stdio.h>
void main(){
int a,b,c;
printf("enter a, b and c value");
scanf("%d%d%d",&a,&b,&c);
printf("%d",(a>b)&&(a<c));
printf("%d",(a>b)||(a<c));
printf("%d",!(a>b));
getch();
}
```

o/p

```
enter a,b,c value
10 20 30
0
1
1
```

④ Assignment Operators

Assignment operators are symbols that help us give values to variables. They are used when we want to store a value in a variable.

The 2 types of assignment operators

① Simple assignment operator

② compound assignment operator

① Simple assignment

Simple assignment operator used to give value to a variable. It takes the value on right side & puts (store) it in variable on the left side.

② compound assignment

compound assignment operator do two things at once, firstly they do a math operation like adding, subtracting, multiplying, dividing then they store result back in same variable

operator	Description	Ex
=	Simple assignment	a=10
+=	compound assignment	a+=10 ⇒ a=a+10
-=		a-=10 ⇒ a=a-10
*=		
/=		
%=		

Example:

```
#include<stdio.h>
void main(){
int a,b;
printf("enter a and b");
scanf("%d%d",&a,&b);
printf("%d",a);
printf("%d",a+=b);
printf("%d",a-=b);
printf("%d",a*=b);
printf("%d",a/=b);
printf("%d",a%=b);
getch();
}
```

o/p.

(5) Bitwise operator.

- Bitwise operator work directly with the binary (bit-level) form of numbers
- In computers, numbers are stored as a series of 0s and 1s, called bits
- Bitwise operators helps us manipulate these bits directly which can be very useful for certain type of programming tasks.

operator	Description
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
<<	Left shift
>>	Right shift
~	One's complement.

Bitwise AND → Both input true then only true(1)

T	F	F	1	0	0
T	T	T	1	1	1 → Both true
F	T	F	0	1	0
F	F	F	0	0	0

Bitwise OR → any one input true it is true

Bitwise XOR → same input → F 0
diff input → T 1

Left shift → 5<<1 5 binary form 101
1 binary form
1 shift

before | | 1 | 0 | 1 |

after 1 shifting → | 1 | 0 | 1 | 0 |

5<<2

after 2 shifting | 1 | 0 | 1 | 0 | 0 |

~ negation T→F, F→T

Right shift 5>>1

before | | 1 | 0 | 1 |

after 1 shifting | | | 1 | 0 | 1 discarded

5>>2

after 2 | | | | 1 | 01 discarded

Example: a=20, b=10

```
#include<stdio.h>
voidmain(){
int a=20,b=10,c=10;
print f("%d",a|b);
printf("%d", a&b);
printf("%d", a^b);
printf("%d", a<<1);
printf("%d", b>>1);
printf("%d",~c);
getch();
}
```

a&b	0
a|b	30
a^b	30
a<<1	40
b>>1	
~c	

TERNARY or CONDITIONAL OPERATOR

Ternary operator is also known as conditional operator is a simple and easy way to make decisions in programming. It allows us to choose one value from two options based on a particular condition. [? :]

Syntax : → exp1 ? exp2 : exp3
if exp1 is true, exp2 is evaluated
Otherwise expression3(exp3) is evaluated

Ex:

```
#include<stdio.h>
void main(){
int z;
z= (5>3) ? 1:0;
printf("%d",z);
getch();
}.
```

Special operator.

1. comma (,) - It is used as seperator for variables for ex: a=10, b=20
2. Address (&) - It get address of variable
3. size of ( ) - It is used to get the size of a data type of a variable in bytes.

integer (2) byte - 16 bit processor
(4) byte - 32

Expression

combination of operator & operands / variables or const

operator precedence

1. ( )
2. [ ]
3. - unary minus
4. ++ --
5. ~
6. !
7. size of
8. (type)

(3–8: right to left)

9. multiplication, division, Modulus
10. Addition, Sub
11. Left shift, right shift
12. Less than, Less than or equal, greater than, Greater than or equals, is equal to, not equal

Operator precedence and associative:

- The concept of operator precedence and associativity in C helps in determining which operators will be given priority when there are multiple operators in the expression.
- It is very common to have multiple operators in C language and the computer first evaluate the operator with higher precedence.
- It helps to maintain the ambiguity of the expression and helps us in avoiding unnecessary use of parenthesis.

Expression - An expression is a combination of operators, constants and variables

- An expression may consist of one or more operators operands, and zero or more operators to produce a value.

result = a + b = c

(a, b: operands; = , +: operators)

Precedence Table

Description	Operator	Associativity
1. Function Expression	( )	left to right ←
2. Array Expression	[ ]	
3. Unary Minus	-	Right to left ←
Increament / Decreament	++ --	
One's compliment	~	
Negation	!	
Size in bytes	sizeof	
Type cast	(type)	

Multiplication	*	Left to Right ←
Division	/	
modulus	%	
Addition	+	
subtraction	-	
Left shift	<<	Left to right
Right shift	>>	Left to right
Less than	<	Left to right
Less than or equal to	<=	
Greater than	>	
greater than or equal to	>=	

Type Conversion

- The term 'type casting' in C is the process of converting one data type to another.
- It is also known as 'type conversion'
- Type conversion is performed by compiler.

Two types of Type Conversion

1. Implicit type conversion
2. Explicit type conversion

1. Implicit type conversion

- It is also called as 'automatic Type conversion'
- There are certain times when the compiler does the conversion on its own (implicit type conversion), so that the data types are compatible with each other.

ex:1.
```
#include<stdio.h>
int main(){
int i=17;
char c='c'; /*ascii value is 99*/
int sum;
sum=i+c;
printf("value of sum %d\n",sum);
}
```

o/p
value of sum : 116

ex 2:
```
#include<stdio.h>
int main(){
int i=17;
char c='c', /*ascii value is 99*/
float sum;
sum = i+c;
printf(" value of sum %f\n ",sum);
}
```

output
value of sum 116.00000

2. Explicit Type Conversion

- This process is also called as Type casting & it is user defined
- Here user can typecast the result to make it of a particular data type.

Syntax in C programming → (type)expression.

type - indicates the datatype to which the final result is converted.

Example 1:

```
#include<stdio.h>
int main(){
int x=10, y=4;
float z;
z=(float)x/y;
printf("%f", z);
return 0;
}
```

output
2.500000

Example 2:

```
#include<stdio.h>
int main(){
float x=1.2;
int sum;
sum=(int)x+1;
printf("sum=%d", sum);
return 0;
}
```

o/p
2

Ex:-

(1) $10*3+6-2 = 30+6-2 = 36-2 = 34$.

(2) $2-5+6-2\%2 = -3+6-0 = 3-0 = 3$

(3) $12/6*3+8. = (2*3)+8 = 6+8 = 14$.

(4) $x=(2+8)*4$.
$x=(10)*4 = 40$.

(5) $x=5+7\%2. = 12\%2 = 0$

(6) $x=1+(2*3/4) = 3*3/4 = \frac{9}{4}$

(7) $z=(9-(3+2))\%2$
$=(9-(5))\%2$
$=4\%2$
$=0$.

(8) $z = 3*((5\%4)*(7+(3/2))/(6+3))$
$= 3*((1)*(7+(1.5)/(9)))$
$= 3*(8.5)/(9)$
$= 2.83$

```
#include<stdio.h>
int main(){
char x='x';
int sum;
sum=(char)x;
printf("%d", sum);
return 0;
}
```

o/p
120

Unit 3

CONTROL - STATEMENT
[Branching and Looping]

Introduction:

Instructions of a programs are executed either
→ (1) Sequential manner
→ (2) Branching

C language supports the following decision making statements
→ (1) If statement
→ (2) Switch statement
→ (3) conditional operator statement
→ (4) goto statement

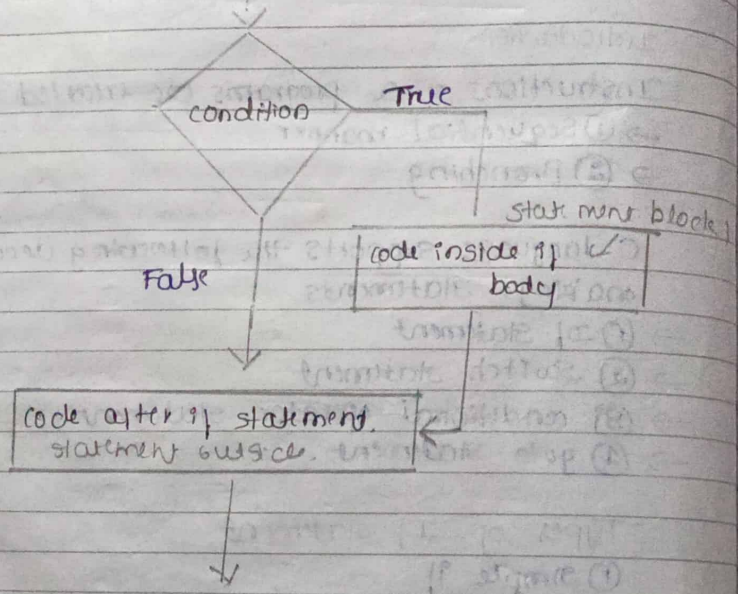
Type of If statement
(1) Simple if
(2) if-else
(3) Nested if else
(4) else if ladder.

① Simple if statement

- It is used to control flow of execution of statement
- It is one way decision making statement and used in conjunction with an expression

Syntax:-
```
if (expression)
{
  statement block;
}
statement outside;
```

Flowchart.



Example for - Simple if

```
#include<stdio.h>
void main()
{
int x;
float y;
printf("Enter x & y value");
scanf("%d%f", &x, &y);
if (x>y)
{
printf("x is a greater than y");
}
}
```

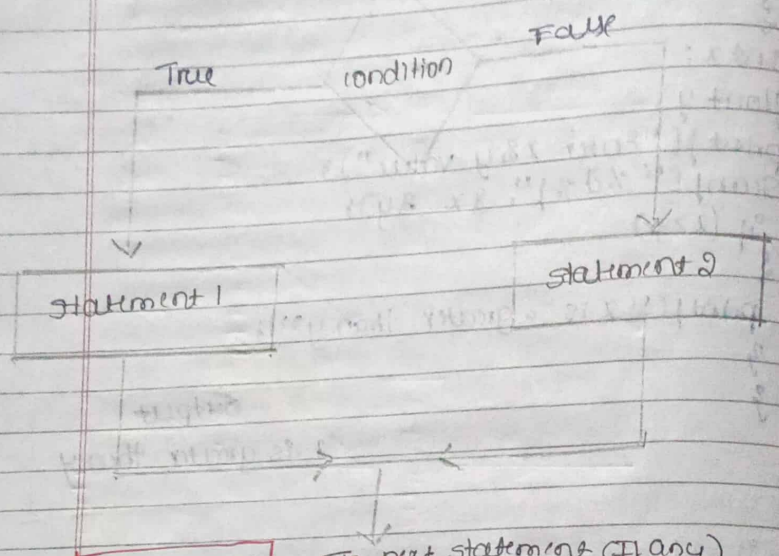
output
x is greater than y

② if-else (even or odd no)

- It is also called as two-way branching
- It is used when there are alternative statements need to be executed based on the condition

Syntax:-
```
if (expression)
{
  Statement block 1;
}
else
{
  Statement block 2;
}
```

Flowchart - if else.



Ex:- Lab program 7

Write a C program to find even or odd numbr.

```
#include<stdio.h>
#include<conio.h>
void main()
{
int num;
printf(" enter any number\n");
scanf("%d", &num);
if(num%2==0)
{
printf("%d is even", num);
}
else
{
printf("%d is odd", num);
}
return 0;
}
```

(3) Nested if-else

If one if-else statement written within another if-else statement such statment is called nested if-else statement.

Syntax -

```
if(condition 1)
{
  if(condition 2)
  {
    statement block1;
  }
else
{
  statement block2;
}
}
else
if (condition 3)
{
 statement block3;
}
else
{
 statement block4;
}
```

Flowchart.

Program for nested if else.

```
#include<stdio.h>
void main()
{
int a,b,c;
printf("enter any 3 numbers\n");
scanf("%d%d%d",&a,&b,&c);
if(a>b)
{
if(a>c)
{
printf("a is the greatest");
}
else
{
printf("c is the greatest");
}
}
else
{
if(b>c)
{
printf("b is the greatest");
}
else
{
printf("c is the greatest");
}
}
}
```

O/P - 2 diamonds

else-if ladder

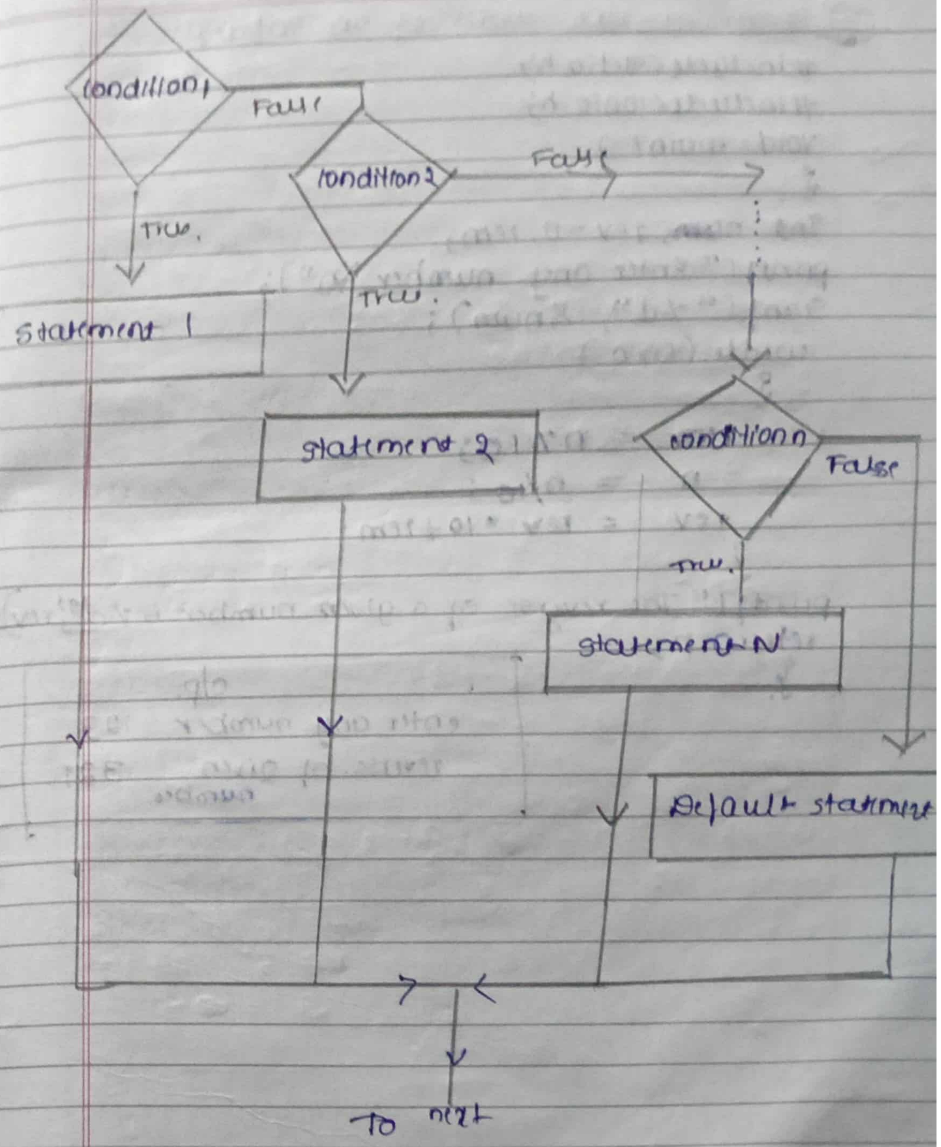
A multiway decision is a chain of if conditions in which the statement associated with an else condition behaves like another if condition

- else if ladder is also called 3 way or multiway decision making statement

```
Syntax :   if (expression 1)
           {
             statement block1;
           }
           else if(expression 2)
           {
             statement block2;
           }
           else if (expression 3)
           {
             statement block 3;
           }
           else
           default statement;
```

lab program-3
ex:- largest of 3 numbers.

Flowchart



Lab Program - 04

④ GCD (Greatest common Divisor) of an integer

```
#include<stdio.h>
int main()
{
int n1, n2;
printf("Enter any two numbers\n");
scanf("%d %d", &n1, &n2);
while (n1 != n2)
{
if (n1 > n2)
  {
    n1 = n1 - n2;
  }
else
  {
    n2 = n2 - n1;
  }
}
printf("gcd = %d", n1);
return 0;
}
```

o/p

```
enter any two number
5  10
gcd = 5
enter any two number
(prime number)
1  5
gcd = 1
```

Switch Statement

- There is one potential problem with the if-else statement which is the complexity of the program increases whenever the number of alternative path increase.
- The solution to this problem is the switch statement.

Rules for switch statement

① In switch statement case labels must be constant or constant expression
② case labels must be unique. No two labels can have the same value.
③ case labels must end with colon.
④ The break statement transfer the control out of the switch statement
⑤ The default label is optional, If present it will be executed when the expression does not find a matching case label.
⑥ There can be at most one default label.
⑦ The default may be placed any where but usually placed at the end.

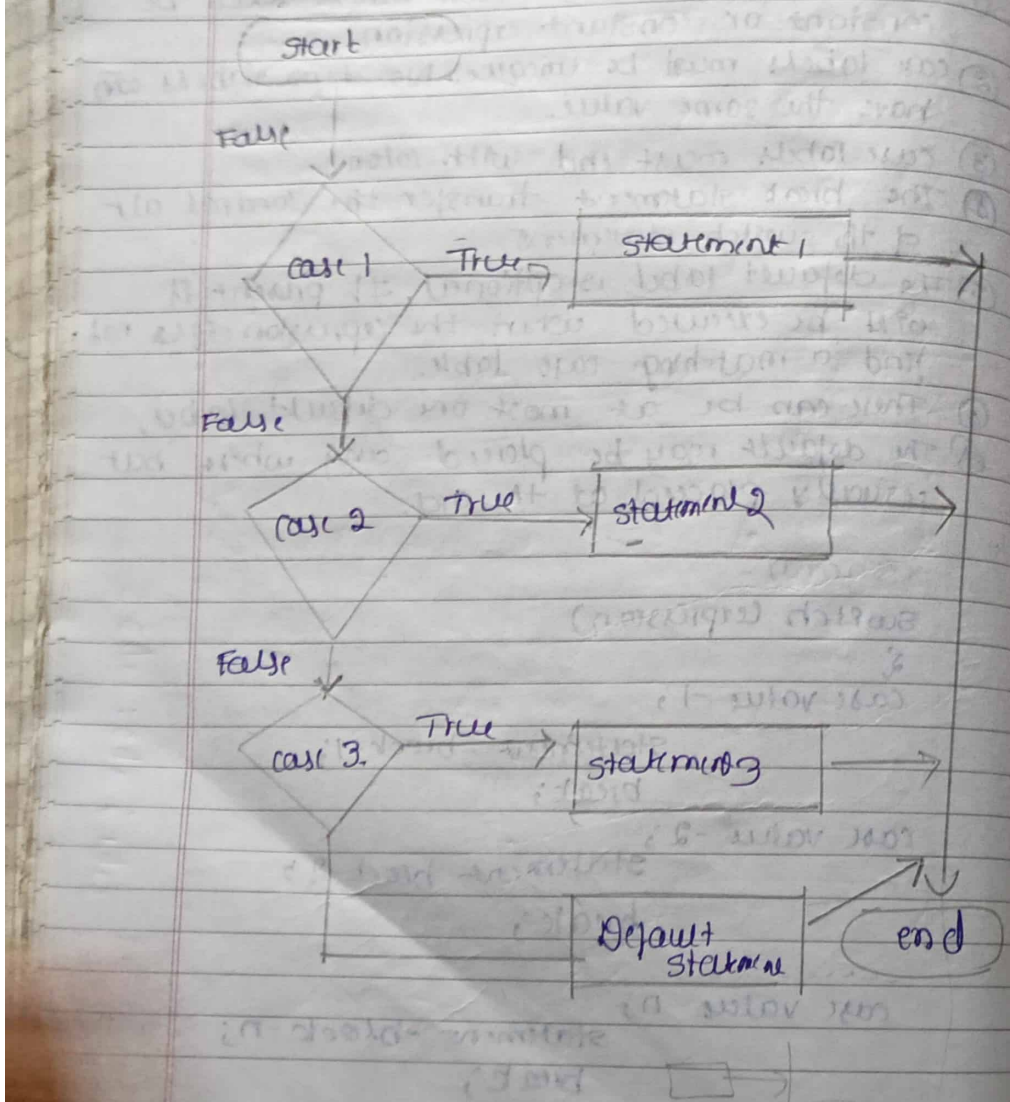
Syntax -

```
switch (expression)
{
case value-1;
        statement-block-1;
        break;
case value-2;
        statement block-2;
        break;
.....
case value n;
        statement-block-n;
        break;
```

default

```
    default statement - block;
        break
}
statement x;
```

Flowchart



Example : 1

```
#include<stdio.h>
void main()
{
int a;
printf("Please enter a no between 1 and 5:");
scanf("%d", &a);
 switch(a)
{
case 1;
printf("You chose one");
break;
case 2; printf("You chose two");
break;
case 3; printf("You chose Three");
break;
case 4; printf("You chose Four");
break;
case 5: printf("You chose Five");
break;
default;
 printf("Invalid choice, Enter a two number
        between 1 and 5");
break;
}}
```

Example -2

```
#include<stdio.h>
int main()
{
 char ch='B'
switch (ch)
{
case 'A': printf("case A");
break;
case 'B': printf("case B");
 break;
 case 'C': printf("case C");
 break;
default : printf("Default");
}
return 0;
}
```

CONDITIONAL OPERATOR (Ternary operator)

[Same as Ternary operator]

conditional operator ? :

exp1 ? exp2 : exp3

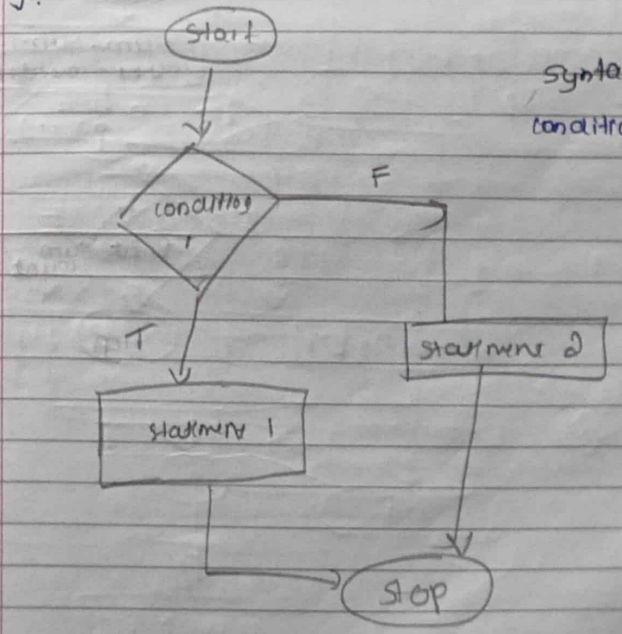
```
flag = (x<6) ? 0 : 1;
```

It is equivalent to

```
if (x<0)
 flag = 0;
else
 flag = 1;
```

Write a C program to find largest of 2 numbers using conditional operator

```
#include <stdio.h>
#include<conio.h>
int main()
{
int num1, num2, max;
printf("Enter two numbers:");
scanf("%d%d", &num1, &num2);
max = (num1>num2) ? num1: num2;
printf("Maximum between %d and %d is
      %d", num1,num2,max);
return 0;
}
```



Syntax

condition ? (expression
expression2

Go-to Statement

- go to statement is a jump statement which is sometimes also referred as unconditional jump statement
- go to statement can be used to jump from anywhere to anywhere within a function (program)

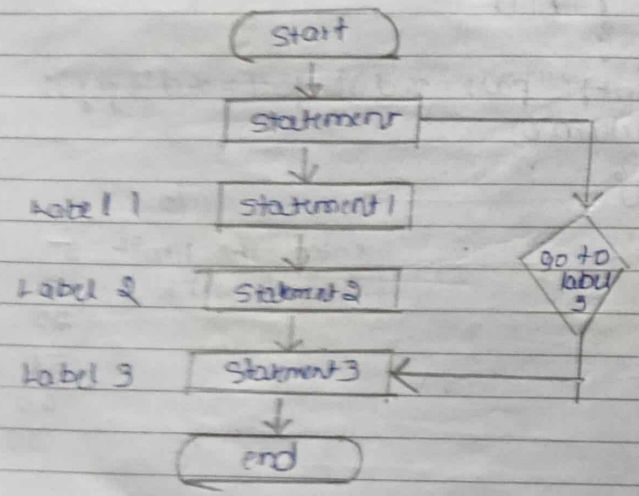
Syntax 1

```
go to label;
...
...
...
label:
statement
```

Syntax 2

```
label:
statement
- - -
- -
go to label;
```

Flowchart




```
/* To print numbers from 1 to 10 using go to statement */

#include <stdio.h>
int main()
{
int number;
number = 1;
repeat :
printf("%d\n", number);
number++;
if (number <= 10)
goto repeat;
return 0;
}
```

o/p

```
number.
1
2
3
4
5
6
7
8
9
10
```

Looping

- Loop executes sequence of statement many times until the stated condition becomes false
- A loop consist of 2 parts
  1. a body of loop
  2. control statement
- control statement is a combination of some conditions that direct the body of the loop to execute until the specified condition becomes false
- The purpose of loop is to repeat the same code number of times

Types

The types of loop

1. while loop → entry controlled.
2. Do while → exit controlled
3. for loop.

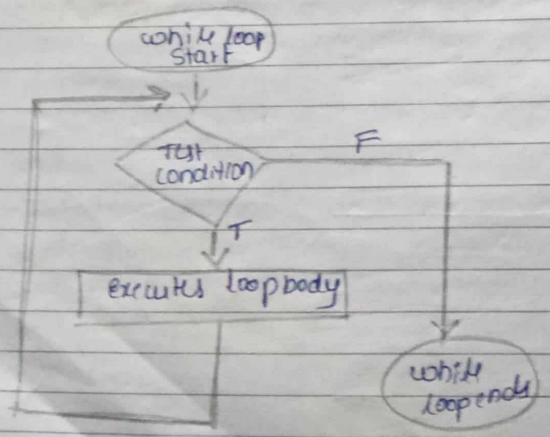
while loop (entry controlled)

while loop in c programming repeatedly executes a target statement as long as a given condition is true

syntax

```
while (condition)
{
  statement 1;
  statement 2;
  . . . .
  statement n;
}
```

Flowchart



Ex: GCD of 2 number Program

write Program reversing digits of an integer

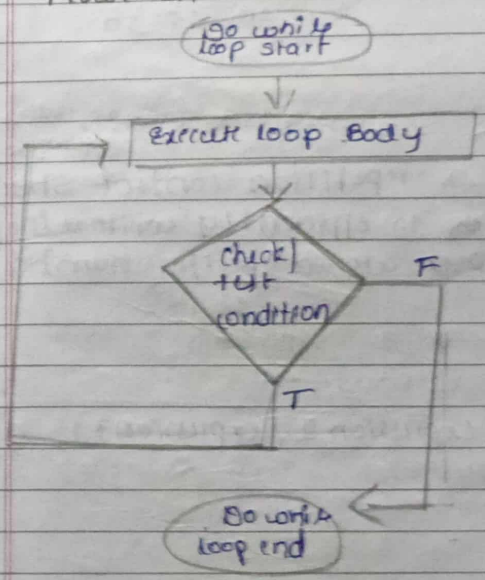
Do while loop → exit controlled.

do while loop is similar to while loop except the fact that is guaranteed to execute at least one time.

Syntax

```
do
{
  statement 1;
  statement 2;
  . . . .
  statement n;
} while (condition);
```

Flowchart



Ex: for do while

```
#include<stdio.h>
void main()
{
int a,i;
 a=5
 i=1;
 do
{
printf("%d\t", a*i);
 i++;
}
while(i<=10);
}
```

o/p

5 10 15 20 25 30 35
40 45 50

For loop → used when both starting & ending point we know

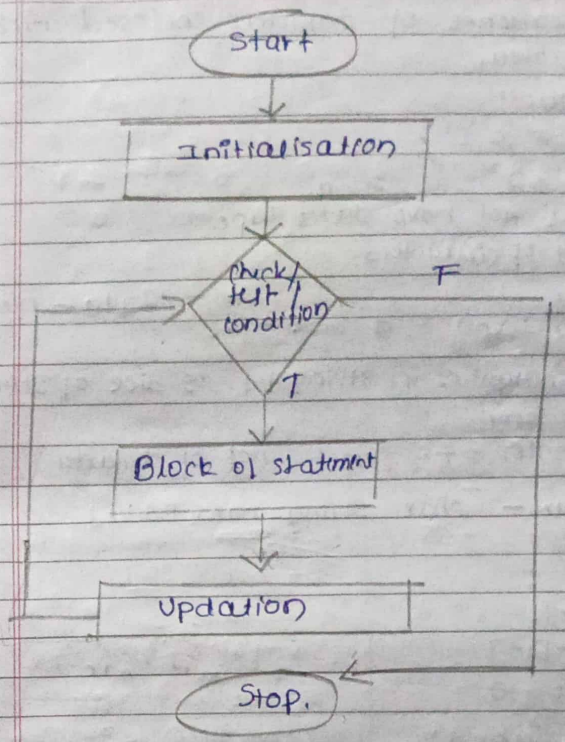
A for loop is a repetition control structure that allows us to efficiently write a loop that needs to execute a specific number of times

Syntax

```
for (expression 1; expression 2; expression 3)
{
  statement 1;
  Statement 2;
  ~~Stat~~.....
  statement n;
}
```

where expression 1 is initialization
expression 2 is condition
expression 3 is updation (increament/decreament)

Flowchart



Ex: Prime number program
factorial no
fibonacci sequence
nth fibonacci
array all programs.

Programs

Nested loop. → one loop write within another loop.

uction)

ameters

parameters

String

Note

String
group / sequence of characters enclosed with double colon.

"abc"
{'a','b','c'}
"1,2"

c does not have string data type.
array of characters.

① "abcd I\0" & size = 6. I\0 – null

no of characters in string + 1 is size of string

~~size leng~~ length = 4. (exact no of character).

syntax – char string-name [size];

char c
→ byte – 1
char c[10]
→ byte = 10

method of storing:
char c[] = "abcd"
char c = {'a', 'b', 'c', '\0'};

c0	c1	c2	c3	c4
1	2	3	4	5

Read
char name[30]
if we use s
scanf("%s", &name)
if we use loop
for (i=0; i<3; i++)
scanf("%c", &name[i])

Display
%s
same as previous
use printf("%s", string)

Char = "c string";
length – 8
it counts space also.

Good (4+1)
size – 5

strcpy – string copy

string handle → headerfile
type <string.h>

→ ① strcat(); string concatenate
joining 2 strings, store in string
strcat (string1, string2)

string1 Hellow string2 world
string1 Hellow. world.
after using strcat
both string 1 & string 2
joined Hello & world
display Hellow world.

→ ② strcmp();
checking whether both are same or not
"a b c" two same
"a b c" string = 0
"a b c" stop checking

a b c	string compare alphabetically
A B C	string1 > string2 +ve value
98	-ve value return
66	
+32	"their", "there"

abcd
abCd
c – 99
C – 67
32

strcmp.
string1 > string2 returns +ve
string1 < string2 -ve value returns

→ ③ strcpy(); string copy
syntax – strcpy (string1, string2)
2nd string 1st string copy
and both string 1 & string 2 display same.

④ string length
strlen():- return the number of characters (length)
exact no of characters it display
BCA1B
length – 5

→ ⑤ string reverse → reverse string
strrev();

→ ⑥ strupr → string upper.
converts all letters to (capital) uppercase

→ ⑦ strlwr()
→ converts lowercase (smaller letters)

1st ③ 2 parameters
5, 6, 7 } 1 parameters

character handling function

toascii() → convert character to 7 bit ASCII character

toupper → convert

to lower →

isalpha → checking character if is alpha or not
if yes = 7. otherwise = 1
0

isdigit → if character is digit 1
else 0

e(1) toascii('b')
98 → (binary 1010)

to upper → B
to lower — b
is alpha — 0

Q. Array defn
explain types 5M – defn Array
one di
2 dim
multi dimen } with syntax

10M → with program expln. no need of 3 dimensional ar.

define string, initialise string, explain string handling function

5M – no need of program.

10M – program

Jumping Out of loops

Sometimes, while executing a loop, it becomes necessary to skip a part of the loop or to leave the loop as soon as certain condition becomes true. This is known as jumping out of loop

→ (1) break statment
→ (2) continue statment

Break statment → it is used to exit a loop.
when break statment is encountered inside a loop, the loop is immediately exited and the program continues with the statment immediately following the loop

Syntax:

```
while (condition check)
{
  statement-1;
  statement-2;
  if (some condition)
  {
    break;
  }
  statment-3;
  statement-4;
}
```

Jump → Jump out of the loop, no matter how many cycles are left, loop is exited

Example for jumping out of loop

```
#include<stdio.h>
int main()
{
int num=5;
while (num>0)
 {
  if (num==3)
   break;
  printf("%d\n", num);
   num--;
 }
}
```

output
5 4

Continue statement → continue statement skips current iteration & jumps next iteration.

It causes the control to go directly to the test condition and then continue the loop process. On encountering continue, cursor leave the current cycle of loop and starts with the next cycle.

Syntax:

```
while(condition check)
 {
  statement 1;
  statement 2;
  if (some condition)
  {
  continue
  }
  statement 3;
  statement 4;
 }
```

Jumps to the

Jump to the next cycle directly

not executed for the cycle of loop in which continue is executed

Example:

```
#include<stdio.h>
int main()
{
int nb=7;
while(nb>0)
{
 nb--;
 if(nb==5)
 continue;
 printf("%d\n", nb);
}
}
```

output
6
4
3
2
1

table multiplication

go-to statement

ARRAY →

- An array is a collection of data that holds fixed number of values of same type

or

Array is a group of ~~elem~~ homogenous elements with same name & same data type

- The size and type of array cannot be changed after its declaration

some examples where the concept of array can be used.

① list of temperature recorded every hour in a day or month or a year
② list of employees in organisation
③ list of products and their cost sold by a stuans
④ test score of a class of student

How to declare an array in C

Syntax

datatype arrayname[size];

Elements of an array and how to access them

We can access elements of an array by index or subscript value.

Key

Key points

- Array have '0' has first index not 1.
  Ex. int marks[3];
  In this example first element is mark[0]

- The size of an array is n, to access last element (n-1) index is used
  ex: mark[2] The size of an array is n. index of last element is (n-1) index

- Suppose the starting address of marks of [0] is 2120 then the address of mark[1] 2122
  mark[2] 2124
  its because size of int is 2 byte

How to initialize an array → using for loop flower bracket

It is possible to initialize an array during declaration for example.

method 1: int marks[3] = {1,2,3};

method 2: int marks[] = {1,2,3};

marks[0] = 1
marks[1] = 2
marks[2] = 3

Types of Array

3 types

① one dimensional → one subscript (index) value — one for loop
② two dimensional → 2 subscript value, 2 for loop
③ Multi dimensional → 3 sub 3

One dimensional Array

A list of elements can be given using one variable name and only one subscript such variable is called one dimensional array

Syntax:- datatype array name[size];

Initialisation of one dimensional array:

- After an array is declared its elements must be initialised
- In c programming an array can be initialised at either of following stages:

  (1) At compile time
  (2) At run time

(1) compile time initialisation (before run)

Syntax:

datatype arrayname[size] = {list of values};

The values in the list are seperated by commas

Ex: int number[5] = {1,2,3,4,5};

number[0] = 1
number[1] = 2
number[2] = 3
number[3] = 4
number[4] = 5.

- If the number of values in the list is less than the number of elements then only that many elements will be initialised the remaining elements will be said to be 0 automatically

- Remember if you have more initializer than the declared size, the compiler will produce an error.

(2) Run time initialization. (after run) using for loop

An array can be explicitly initialised at run time

Ex: int number[5]

consider the following segment of c program

```
int number[5];
for(i=0;i<5;i++)
{
scanf("%d", &number[i]);
}
```

Ex:

```
#include<stdio.h>
int main()
{
int number[5];
printf("Enter array elements");
for(i=0;i<5;i++)
{
scanf("%d", &number[i]);
}
printf("Elements are");
for(i=0;i<5;i++)
{printf("%d", number);
}
return 0;
}
```

Two Dimensional Array

A list of elements can be given using one variable name and 2 subscript such variable is called as 2 dimensional array. An array with 2 dimension is called ~~2D~~ 2D

Syntax:

datatype arrayname[row size] [column size];

Example: int a[3][4];

2 dimensional array a which contain 3 rows and 4 columns are consider as follows

	column 0	column 1	column 2	column 3
row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Initialisation of two dimensional array

2 dimensional array may be initialised by specifying bracketed values for each row

Ex: int a[3][4] = {{1,2,3,4}{5,6,7,8}{9,10,11,12}};

Nested bracket indicates intended row.

The following initialisation is equivalent to previous example.

int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};

Program 2 Dimensional Array

```
#include<stdio.h>
int main()
{
int number[5][2], i, j;
printf("Enter array elements");
for(i=0;i<5;i++)
{
for(j=0;j<2;j++)
scanf("%d", &number[i][j]);
printf("elements are");
for(i=0;i<5;i++)
{
for(j=0;j<2;j++)
printf("%d", number[i][j]);
}
}
return 0;
}
```

Multidimensional Array

An array with more than 2 subscript is called Multi dimensional array

Syntax

datatype arrayname[d1][d2]....[dn];

Ex:-

int cube[3][2][3]

Important (all)

history of C

v.v.Imp features of C

v.v.Imp Structure of C, ex:- block diagram executing C program

2nd v.v.Imp C tokens → 5 token. all Imp.

quantifier rules.

datatypes → all Imp.

10M - all

5M - defination

Data type modifier

I/p & o/p

v.v. Imp formatted I/p → program

informatics → syntax

defination

example.

define operator, increment, decrement

Imp

v.v.Imp binary operator

operator symbol

precedence.

statement syntax

flowchart

program.

define array

one dimensional

two dimensional

function →

A function call itself → recursion

Unit-2: Introduction to C

Date 22/9/24.

Chapter 1: Overview of C.

History of C Language.

- 'C' is one of the most popular general purpose and procedural programming language.
- Dennis Ritchie, A computer scientist could identify the gaps and get the best feature from both 'B' and 'BCPL' [Basic combined programming language] ##.
- Hence C was introduced in 1972 at (AT&T) AT&T (AMERICAN Telephone & Telegram) which is located in USA.
- 'C' is one of the most popular computer language today because of its structure and machine independent feature
- Sooner 'C' started spreading its roots into various IT Domain when programmers face troubles with B & BCPL. 'C' served It's purpose and successfully solved the rooted in problems faced by the major part of traditional programming languages.

Importance of C

- A complete operating system designs using C that's when the early windows & unix operating system came into existence.
- Unix was the first ever fully functional operating system to be completely design, and develop using C.
- Google
- C language helps to do application level programming
- Google, mozilla firefox and My SQL are the popular example are the another key to its success it became user friendly and

Memory representation in one dimensional array

each elements can be stored one after another in contiguous manner

* elements can be stored in one after another
* accessed using index value
* fixed size ⇒ can not change.

```
int a[5] = {10, 20, 30, 40, 50};
```

assume starting address 1000

Index	value	Address	integer 2byte
a[0]	10	1000	1000
a[1]	20	1002	1002
a[2]	30	1004	1004
a[3]	40	1008	1008
a[4]	50	1010	1010.

* Two dimensional array memory Representation

```
int a[3][4] = {{10 20 30 40}
               {50 60 70 80}
               {90 100 110 120}};
```

(continuous manner)

a[0][0]	10	1000
a[0][1]	20	1002
a[0][2]	30	1004
a[0][3]	40	1006
a[1][0]	50	1008
a[1][1]	60	1010
a[1][2]	70	1012
a[1][3]	80	1016
a[2][0]	90	1018
a[2][1]	100	1020
a[2][2]	110	1022
a[2][3]	120	1024

control strings. → format specifier

Escape sequence. → special character in strings.

Escape sequence	Meaning
\n	new line
\t	Tab
\\	Backslash → special character use of.
\"	Double quote
\'	single quote.
\b	backspace.